



Exponent Vaults

Security Assessment

May 29th, 2026 — Prepared by OtterSec

Ajay Shankar Kunapareddy

d1r3wolf@osec.io

Xiang Yin

soreatu@osec.io

Table of Contents

Executive Summary	4
Overview	4
Key Findings	4
Scope	6
Findings	7
Vulnerabilities	8
OS-EXV-ADV-00 SY Valued in Incorrect Base Asset	16
OS-EXV-ADV-01 YT Complement Pricing Assumes Matching Base Asset	17
OS-EXV-ADV-02 Dust Withdrawals Permanently Block Manager Actions	19
OS-EXV-ADV-03 Stale Backlog Timestamp Blocks Managers	20
OS-EXV-ADV-04 Proposal Rejection Uses Live Threshold	21
OS-EXV-ADV-05 Chainlink and Reflect Prices Skip Decimal Adjustment	22
OS-EXV-ADV-06 Stale Withdrawal Fill Capacity After Token Changes	23
OS-EXV-ADV-07 Lack of Foundational-Policy Guard on Policy Removal	24
OS-EXV-ADV-08 Permissionless Global Price Registry Initialization	25
OS-EXV-ADV-09 Singleton Price Registry Reinitialization Takeover	26
OS-EXV-ADV-10 Missing Mutable Vault in Proposal Lifecycle	27
OS-EXV-ADV-11 Unsafe Price Type Migration	28
OS-EXV-ADV-12 Utilization of Incorrect Confidence Bound in PythInverse	29
OS-EXV-ADV-13 PT/YT Pricing Utilizes Linear Discounting	31
OS-EXV-ADV-14 Hylo Collateral Ratio Uses Pool Balance	32
OS-EXV-ADV-15 Sentinel Escalation Ignores Withdrawal Carryover	33

OS-EXV-ADV-16	Earlier Opt-Out Withdrawal Deadline Ignored	35
OS-EXV-ADV-17	Opt-Out Votes Bypass Withdrawal Blocking	36
OS-EXV-ADV-18	FillWithdrawal Leaves Post-Fill AUM Stale	37
OS-EXV-ADV-19	Withdrawal Fees Rounded Away By Splitting	38
OS-EXV-ADV-20	Double-Incrementing of Kamino Reserve Price Usage	39
OS-EXV-ADV-21	AUM-Sensitive Checks Use Stale Financials	40
OS-EXV-ADV-22	MarketOffer Validation Omits AUM Diff	41
OS-EXV-ADV-23	Initial Token Entries Lack Account Validation	42
OS-EXV-ADV-24	Wrapper Policy Updates Skip Configuration Validation	43
OS-EXV-ADV-25	Locked Withdrawal AUM Uses Raw LP Supply	44
OS-EXV-ADV-26	Permissionless Vault Initialization may Pin Shared Prices	45
OS-EXV-ADV-27	Direct Entrypoints Bypass Role Semantics	46
OS-EXV-ADV-28	Allocator Wrappers Allow Non-Additive Live Changes	47
OS-EXV-ADV-29	Delegated Accounts Accepted As Withdrawal Sources	48
OS-EXV-ADV-30	Ability of Managers to Remove the Last Sentinel	49
OS-EXV-ADV-31	Zero Mint Bypasses Price Validation	50
OS-EXV-ADV-32	Pre-Funded PDAs Block Account Creation	51
OS-EXV-ADV-33	AddPrice Accepts PT/YT Without APY Metadata	52
OS-EXV-ADV-34	Failure to Back Active Proposal Votes by Eligibility Checkpoint	53
OS-EXV-ADV-35	Executed Proposals Bypass Opt-Out Withdrawals	54
OS-EXV-ADV-36	Configuration Updates may Retroactively Extend Queued Withdrawals	55
OS-EXV-ADV-37	Policy Manager Update Drops Existing Metadata	56
General Findings		57

OS-EXV-SUG-00 Missing Validation Logic	58
OS-EXV-SUG-01 Access Control and Administration Separation	63
OS-EXV-SUG-02 Code Refactoring	65
OS-EXV-SUG-03 Accounting and Price Source Consistency	67
OS-EXV-SUG-04 Redundant/Unutilized Code	69
OS-EXV-SUG-05 Code Maturity	71

Appendices

Vulnerability Rating Scale	72
Procedure	73

01 — Executive Summary

Overview

Exponent Finance engaged OtterSec to assess the `vaults` program. This assessment was conducted between April 7th and May 26th, 2026. For more information on our auditing methodology, refer to [Appendix B](#).

Key Findings

We produced 44 findings throughout this audit engagement.

In particular, we identified two critical vulnerabilities, including one in which SY exposure is valued using orderbook/yield exchange rates denominated in its own base asset, not necessarily the strategy vault's underlying mint. When those assets differ, AUM is computed in the wrong unit, resulting in incorrect vault valuation ([OS-EXV-ADV-00](#)). The second critical issue concerns incorrect YT pricing when the yield base asset differs from the strategy vault's underlying mint, resulting in YT AUM being valued in the wrong unit and materially misstating vault value ([OS-EXV-ADV-01](#)).

Also, Chainlink and Reflect return normalized oracle prices but skip mint-decimal adjustment into the vault's underlying accounting unit, resulting in AUM being mis-scaled if token decimals differ ([OS-EXV-ADV-05](#)), and policies under the vault's Squads settings may be removed, which may permanently disable core vault flows if the vault PDA/signers cannot be restored ([OS-EXV-ADV-07](#)).

Additionally, price initialization logic is permissionless despite controlling a single global ExponentPrices account, enabling an arbitrary caller to initialize the account first and install an unintended manager ([OS-EXV-ADV-08](#)), and if all price managers are removed, price initialization lets the next arbitrary caller re-initialize the live singleton ExponentPrices account and become its manager, creating a protocol-wide takeover path over the global price registry, potentially breaking or redirecting all pricing-dependent flows ([OS-EXV-ADV-09](#)).

We further highlighted an issue in which the price update logic only changes the price type, leaving old interface accounts, cached price data, and freshness metadata in place ([OS-EXV-ADV-11](#)), and an issue regarding the utilization of the inverse upside confidence bound in PythInverse when checking negative inverse-price moves, which may overstate tolerated downside movement and keep a stale higher inverse price instead of accepting a real price decrease ([OS-EXV-ADV-12](#)).

Furthermore, a vault may be initialized with invalid or attacker-controlled token accounts, as vault initialization only validates token-entry mint uniqueness and price IDs, but does not verify token-account ownership or mint consistency ([OS-EXV-ADV-22](#)). Vault initialization also permits a TokenEntry with a zero mint, which disables the terminal mint check, allowing a reserve to be priced with an unrelated

asset's price chain, which breaks reserve-to-price binding, enabling fake reserve valuation, distorted AUM/LP pricing, stuck price references, and withdrawal mismatches ([OS-EXV-ADV-31](#)).

We also advised incorporating validation logic for improved robustness ([OS-EXV-SUG-00](#)), and recommended refactoring the account handling paths ([OS-EXV-SUG-02](#)). We further suggested removing unnecessary and redundant code ([OS-EXV-SUG-04](#)) and ensuring adherence to coding best practices ([OS-EXV-SUG-05](#)).

02 — Scope

The source code was delivered to us in a Git repository at <https://github.com/exponent-finance/exponent-monorepo>. This audit was performed against commit [1d4c189](#).

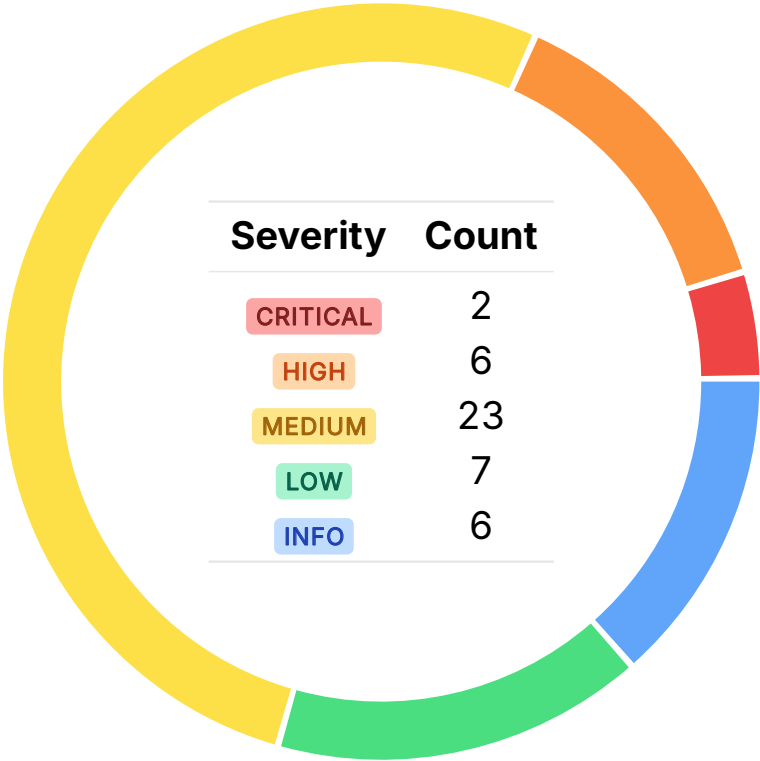
A brief description of the program is as follows:

Name	Description
vaults	A Solana vault-management program creates strategy vaults and tracks deposits, LP shares, withdrawals, strategy positions, prices, AUM, fees, and reserve requirements. It integrates with Squads policies and proposal-based voting flows to constrain manager actions while supporting controlled updates to vault settings, positions, withdrawals, and external strategy execution.

03 — Findings

Overall, we reported 44 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in [Appendix A](#).

ID	Severity	Status	Description
OS-EXV-ADV-00	CRITICAL	RESOLVED ✓	<code>SY</code> exposure is valued using orderbook/yield exchange rates denominated in their own base asset, not necessarily the strategy vault's <code>underlying_mint</code> . When those assets differ, AUM is computed in the wrong unit, resulting in incorrect vault valuation.
OS-EXV-ADV-01	CRITICAL	RESOLVED ✓	<code>YT</code> is priced as <code>1 - pt_price</code> , which is only correct when the yield base asset matches the strategy vault's <code>underlying_mint</code> . If those assets differ, <code>YT</code> AUM is valued in the wrong unit and may materially misstate vault value.
OS-EXV-ADV-02	HIGH	RESOLVED ✓	Dust queued withdrawals may become unfillable when <code>base_out_from_lp</code> rounds their base output to zero and reverts. The residual LP remains pending, potentially resulting in overdue withdrawal state to block manager actions indefinitely.
OS-EXV-ADV-03	HIGH	RESOLVED ✓	A single global pending withdrawal backlog timestamp may become stale after the earliest withdrawal is cleared while later withdrawals remain pending, resulting in the blocking of manager actions before the remaining withdrawal is actually overdue.

OS-EXV-ADV-04	HIGH	RESOLVED ✓	<code>finalize_proposal</code> reports the frozen rejection threshold but decides the outcome utilizing the vault's live threshold configuration. If <code>rejection_threshold_bps</code> changes during voting, a proposal may be approved or rejected under a different threshold than the one snapshotted at activation.
OS-EXV-ADV-05	HIGH	RESOLVED ✓	Chainlink and Reflect return normalized oracle prices but skip mint-decimal adjustment into the vault's underlying accounting unit, resulting in AUM to be mis-scaled if token decimals differ.
OS-EXV-ADV-06	HIGH	RESOLVED ✓	<code>WithdrawalAccount</code> size is fixed using the vault's token-entry count at queue time. If token entries are later added or remapped, older withdrawals may lack space to record required fill slots, rendering them unfillable or partially stuck.
OS-EXV-ADV-07	HIGH	RESOLVED ✓	<code>apply_remove_policy</code> may remove any policy under the vault's Squads settings, including foundational FILL or TRANSFER policies. Removing these required policies can permanently disable core vault flows if the vault PDA/signers cannot be restored.
OS-EXV-ADV-08	MEDIUM	RESOLVED ✓	<code>initialize_prices</code> is permissionless despite controlling a single global <code>ExponentPrices</code> account, so an arbitrary caller may initialize the account first and install an unintended manager.

OS-EXV-ADV-09	MEDIUM	RESOLVED ✓	Currently, if all price managers are removed, <code>initialize_prices</code> lets the next arbitrary caller re-initialize the live singleton <code>ExponentPrices</code> account and become its manager. That creates a protocol-wide takeover path over the global price registry, potentially breaking or redirecting all pricing-dependent flows.
OS-EXV-ADV-10	MEDIUM	RESOLVED ✓	These proposal instructions mutate <code>live_proposal_count</code> , but <code>vault</code> is not marked <code>mut</code> , so the changes are not persisted. That silently breaks live-proposal tracking.
OS-EXV-ADV-11	MEDIUM	RESOLVED ✓	<code>UpdatePrice</code> only changes the price type, leaving old interface accounts, cached price data, and freshness metadata in place. As a result, existing price IDs may serve stale or semantically invalid prices, especially for <code>PT</code> / <code>YT</code> types that require APY parameters.
OS-EXV-ADV-12	MEDIUM	RESOLVED ✓	<code>PythInverse</code> utilizes the inverse upside confidence bound even when checking negative inverse-price moves, which may overstate tolerated downside movement and keep a stale higher inverse price instead of accepting a real price decrease.
OS-EXV-ADV-13	MEDIUM	RESOLVED ✓	<code>pt::read_price</code> values <code>PT</code> / <code>YT</code> with linear discounting while orderbook/CLMM utilize exponential discounting. This creates systematic cross-component price inconsistency that may misstate AUM and downstream vault accounting.

OS-EXV-ADV-14	MEDIUM	RESOLVED ✓	<code>compute_nav</code> uses the stability pool's stablecoin balance as the collateral-ratio denominator instead of total stablecoin supply. This may overstate collateralization, select an overly healthy stability mode, and overprice Hylo LP NAV in vault AUM.
OS-EXV-ADV-15	MEDIUM	RESOLVED ✓	<code>can_make_sentinel_manager</code> ignores weekly withdrawal carryover and escalates from the current/next window end directly. Late-window withdrawals may therefore let sentinels gain manager authority before the withdrawal is actually overdue.
OS-EXV-ADV-16	MEDIUM	RESOLVED ✓	<code>add_pending_withdrawal</code> only sets the backlog deadline when the pending set was empty, so a later-added opt-out withdrawal with an earlier backdated due time may not update it. Manager actions may remain unblocked even though that opt-out withdrawal is already overdue.
OS-EXV-ADV-17	MEDIUM	RESOLVED ✓	<code>stake_vote</code> permits <code>VoteChoice::OptOut</code> without checking the vault's withdrawal-blocked flag. If opt-out votes later create withdrawal claims, they bypass the same emergency withdrawal pause enforced on ordinary withdrawals.
OS-EXV-ADV-18	MEDIUM	RESOLVED ✓	<code>fill_withdrawal</code> recalculates AUM before transferring reserves and updating withdrawal state, but does not refresh it afterwards. This may leave <code>vault.financials.total_aum()</code> stale for later AUM-sensitive checks and limits.
OS-EXV-ADV-19	MEDIUM	RESOLVED ✓	<code>calculate_lp_fee_split</code> floors the fee per withdrawal fill, so small fills may round the fee to zero. By splitting a large withdrawal into many small chunks, users may reduce or bypass the intended withdrawal fee.

OS-EXV-ADV-20	MEDIUM	RESOLVED ✓	Kamino reserve price mappings may increment price usage during validation and then again via <code>update_kamino_reserve_price_pointers</code> , overstating usage counters, preventing unused prices from being removed and corrupting global price accounting.
OS-EXV-ADV-21	MEDIUM	RESOLVED ✓	AUM-sensitive checks read cached <code>vault.financials.total_aum()</code> before refreshing AUM, so stale accounting may incorrectly allow or reject settings, policy, and position actions. Recalculate AUM before such gates and after AUM-changing mutations.
OS-EXV-ADV-22	MEDIUM	RESOLVED ✓	Orderbook <code>market_offer</code> validations return <code>aum_diff == 0</code> even though the trade may mutate tracked vault balances. This may enable value-changing market offers to bypass AUM-change limits and withdrawal-protection checks.
OS-EXV-ADV-23	MEDIUM	RESOLVED ✓	<code>initialize_vault</code> validates only token-entry mint uniqueness and price IDs, but does not verify token-account ownership or mint consistency. A vault may therefore be initialized with invalid or attacker-controlled token accounts that later break transfers and accounting.
OS-EXV-ADV-24	MEDIUM	RESOLVED ✓	Wrapper/proposal policy paths forward <code>PolicyConfig</code> to shared apply helpers without validation, unlike direct admin handlers, allowing malformed Squads policies to be installed through alternate entrypoints.

OS-EXV-ADV-25	MEDIUM	RESOLVED ✓	<code>validate_interaction</code> converts locked withdrawal LP to AUM using raw <code>mint_lp.supply</code> instead of effective LP supply. This may understate withdrawal-protected AUM and allow managed interactions to deploy or move more value than reserve constraints should permit.
OS-EXV-ADV-26	MEDIUM	RESOLVED ✓	<code>initialize_vault</code> is permissionless but registers token-entry price references in the shared global <code>ExponentPrices</code> registry, enabling an attacker to create arbitrary vaults to pin price IDs and prevent their later removal from the global registry.
OS-EXV-ADV-27	MEDIUM	RESOLVED ✓	Direct <code>manage_vault_settings</code> and <code>manager_update_position</code> rely only on the vault PDA signing, while wrapper flows enforce manager/allocator roles. Any route that may make the vault sign direct instructions will bypass wrapper role restrictions and mutate vault state through the weaker path.
OS-EXV-ADV-28	MEDIUM	RESOLVED ✓	<code>wrapper_manage_vault_settings</code> allows allocator callers to execute most settings and position mutations, only blocking role-member changes. Without a strict allocator-specific allowlist, a compromised allocator may perform broader non-additive vault modifications than intended.
OS-EXV-ADV-29	MEDIUM	RESOLVED ✓	<code>execute_withdrawal</code> accepts token sources delegated to the vault PDA, not just vault-owned accounts. Stale or misconfigured delegations can therefore let withdrawals consume funds from unintended token accounts.

OS-EXV-ADV-30	MEDIUM	RESOLVED ✓	<code>remove_role_member</code> prevents removing the last manager but allows removing the last sentinel. A manager may therefore disable the vault's emergency recovery and overdue-withdrawal escalation path by eliminating all sentinels.
OS-EXV-ADV-31	LOW	RESOLVED ✓	A zero <code>TokenEntry.mint</code> disables <code>validate_price</code> 's terminal mint check, so a reserve may be priced with an unrelated asset's price chain. This breaks reserve-to-price binding, enabling fake reserve valuation, distorted AUM/LP pricing, stuck price references, and withdrawal mismatches.
OS-EXV-ADV-32	LOW	RESOLVED ✓	Deterministic account creation utilizes <code>create_account</code> , which will fail if an attacker pre-funds the target PDA with dust lamports. This will block specific price-interface, LP mint, or token-account seeds, resulting in denial-of-service scenario.
OS-EXV-ADV-33	LOW	RESOLVED ✓	<code>AddPrice</code> accepts <code>PriceType::Pt</code> and <code>PriceType::Yt</code> but has no <code>implied_apy_bps</code> field required for <code>PT</code> / <code>YT</code> pricing, creating an inconsistent initialization path.
OS-EXV-ADV-34	LOW	RESOLVED ✓	<code>stake_vote</code> utilizes live LP balances at vote time rather than proposal-specific eligibility checkpoints. This allows post-activation LP to vote on active proposals, making the voting set inconsistent with the activation-time rejection threshold.

OS-EXV-ADV-35	LOW	RESOLVED ✓	<code>unstake_vote</code> queues opt-out withdrawals only while a proposal is <code>Approved</code> . After it becomes <code>Executed</code> , opt-out voters get LP returned normally. This lets execution timing bypass the intended opt-out withdrawal semantics.
OS-EXV-ADV-36	LOW	RESOLVED ✓	Queued withdrawals rely on live withdrawal settings, so managers may retroactively extend execution or cancellation delays by updating withdrawal-period configuration after users have already queued withdrawals.
OS-EXV-ADV-37	LOW	RESOLVED ✓	<code>update_policy_manager</code> should only replace the policy signer, but it reconstructs <code>SpendingLimit</code> policy payloads and may drop metadata such as <code>accumulate_unused</code> or settings-based expiration, unintentionally changing policy semantics during sentinel takeover.

SY Valued in Incorrect Base Asset

CRITICAL

OS-EXV-ADV-00

Description

`calculate_aum_in_base` treats `raw_orderbook.last_sy_exchange_rate` as the `SY` price used for strategy-vault AUM, but that exchange rate is only denominated in the orderbook/core vault's base asset, not necessarily in the strategy vault's `underlying_mint`. If the orderbook base asset and the strategy vault underlying differ, `add_amount_in_base(..., &price_sy)` converts `SY` exposure into the wrong unit and overstates or understates AUM depending on the relative asset prices. The same class of issue applies to yield-position valuation when `last_seen_sy_exchange_rate` is used as a direct valuation price in `YieldPositionEntry`.

Remediation

Store and validate explicit `SY` price IDs for orderbook and yield positions, and value `SY` exposure through the vault's underlying price path.

Patch

Resolved in [PR#3802](#).

YT Complement Pricing Assumes Matching Base Asset CRITICAL OS-EXV-ADV-01

Description

In the current implementaion, **YT** price is derived as the complement of **PT** price. `pt::read_price` derives **YT** value as `1 - pt_price`, implicitly assuming that **PT** and **YT** together settle to one unit of the same asset used as the strategy vault's `underlying_mint`. That assumption is only valid when the yield base asset, PT quote unit, and vault accounting unit are aligned. If the Exponent Core vault's yield base differs from the strategy vault underlying, the complement value represents residual yield in the wrong unit of account, so **YT** exposure may be mispriced in AUM.

```
>_ libraries/exponent_vaults_integrations/src/price_sources /pt.rs
```

RUST

```
pub fn read_price(
    last_updated_slot: u64,
    interface_accounts: &[Pubkey],
    accounts: &[AccountInfo],
    clock: &Clock,
    mint_decimals: Option<(u8, u8)>,
    implied_apy: f64,
    yt: bool,
) -> Result<Option<Number>> {
    [...]
    let seconds_to_maturity = (maturity_timestamp - current_timestamp) as f64;
    let years_to_maturity = seconds_to_maturity / (365.25 * 24.0 * 60.0 * 60.0);
    let discount_factor = 1.0 + (implied_apy * years_to_maturity);
    let pt_price = 1.0 / discount_factor;
    let scaled_pt_price = (pt_price * PRICE_SCALE as f64) as u128;
    let pt_price = Number::from_ratio(scaled_pt_price, PRICE_SCALE);
    let price = if yt {
        Number::ONE
            .checked_sub(&pt_price)
            .ok_or(ErrorCodeVaults::MathOverflow)?
    } else {
        pt_price
    };
    [...]
}
```

Remediation

Value **YT** exposure using explicit **SY** and **PT** prices, or reject generic derived **YT** pricing when the required base-asset relationship cannot be guaranteed.

Patch

Resolved in [PR#3803](#).

Dust Withdrawals Permanently Block Manager Actions HIGH OS-EXV-ADV-02

Description

`vault::fill_withdrawal` currently calls `base_out_from_lp` to convert the user's fee-adjusted LP amount into base AUM, but `base_out_from_lp` floors `total_aum * lp_in / lp_supply` and rejects if the result is zero. When a queued withdrawal is reduced to a dust remainder, such as `lp_amount_remaining == 1`, and vault AUM per LP is below one base unit, the pro-rata base output rounds down to zero and the fill reverts with `AmountTooSmall`.

```
>_ exponent_vaults/src/state/vault.rs
```

```
RUST
```

```
pub fn base_out_from_lp(&self, lp_supply: u64, lp_in: u64) -> Result<u64> {  
    [...]  
    let numerator = u128::from(self.total_aum())  
        .checked_mul(u128::from(lp_in))  
        .ok_or(ErrorCodeVaults::MathOverflow)?;  
  
    let base_out = numerator  
        .checked_div(u128::from(lp_supply))  
        .ok_or(ErrorCodeVaults::MathOverflow)?;  
    let base_out = u64::try_from(base_out).map_err(|_| ErrorCodeVaults::MathOverflow)?;  
  
    require!(base_out > 0, ErrorCodeVaults::AmountTooSmall);  
    Ok(base_out)  
}
```

Because the LP remainder cannot be burned through the normal fill path, `lp_amount_remaining`, `lp_locked_for_withdrawal`, and `total_lp_pending_withdrawals` may remain nonzero indefinitely. Once `pending_withdrawal_backlog_due_at` has elapsed, `blocks_manager_actions_for_pending_withdrawals` will keep manager actions blocked even though the remaining claim has no redeemable base value.

Remediation

Allow zero base output for dust withdrawal cleanup while preserving accounting invariants.

Patch

Resolved in [PR#3771](#).

Stale Backlog Timestamp Blocks Managers

HIGH

OS-EXV-ADV-03

Description

In `vault`, `remove_pending_withdrawal` only resets `pending_withdrawal_backlog_due_at` when `total_lp_pending_withdrawals` reaches zero, so the vault keeps a single global backlog timestamp across multiple queued withdrawals. If the earliest withdrawal that established that timestamp is cleared, but a later withdrawal remains pending, the stored deadline may still reflect the already-cleared withdrawal rather than the next live withdrawal's actual executable/backlog time.

```
>_ exponent_vaults/src/state/vault.rs
```

RUST

```
pub fn remove_pending_withdrawal(&mut self, amount: u64) -> Result<()> {
    self.total_lp_pending_withdrawals = self
        .total_lp_pending_withdrawals
        .checked_sub(amount)
        .ok_or(ErrorCodeVaults::MathOverflow)?;
    if self.total_lp_pending_withdrawals == 0 {
        self.pending_withdrawal_backlog_due_at = 0;
    }
    Ok(())
}

pub fn blocks_manager_actions_for_pending_withdrawals(&self, current_time: u32) -> bool {
    self.total_lp_pending_withdrawals > 0
        && self.pending_withdrawal_backlog_due_at > 0
        && current_time >= self.pending_withdrawal_backlog_due_at
}
```

`blocks_manager_actions_for_pending_withdrawals` then checks only `total_lp_pending_withdrawals > 0` and `current_time >= pending_withdrawal_backlog_due_at`, so the stale timestamp may result in manager actions to be blocked too early.

Remediation

Avoid utilizing a stale single backlog timestamp to block manager actions. Instead, recompute it from the remaining pending withdrawals.

Patch

Resolved in [PR#3772](#).

Proposal Rejection Uses Live Threshold HIGH

OS-EXV-ADV-04

Description

`finalize_proposal` captures `rejection_required_snapshot()` for reporting, but the actual approval/rejection decision calls `vault.proposal_vote_config.is_proposal_rejected`. That recomputes the threshold from the vault's current `rejection_threshold_bps`, and not from the threshold frozen when the proposal was activated. Since `rejection_threshold_bps` may be changed while the proposal is active, finalization may apply a higher or lower rejection threshold than voters were operating under.

```
>_ exponent_vaults/src/instructions/proposals/finalize_proposal.rs
```

RUST

```
pub fn handler<'info>(ctx: Context<'_, '_, '_, 'info, FinalizeProposal<'info>>) -> Result<()> {  
    [...]  
    let result = if ctx  
        .accounts  
        .vault  
        .proposal_vote_config  
        .is_proposal_rejected(effective_reject_votes, total_lp_supply)?  
    {  
        ProposalStatus::Rejected  
    } else {  
        ProposalStatus::Approved  
    };  
    [...]  
}
```

Remediation

Finalize with `effective_reject_votes >= proposal.rejection_required_snapshot()`.

Patch

Resolved in [PR#3779](#).

Chainlink and Reflect Prices Skip Decimal Adjustment HIGH OS-EXV-ADV-05

Description

`chainlink::read_price` normalizes the oracle answer only by the feed's own decimal exponent, returning a human-denominated exchange rate such as `price.value / 10price.decimals`. It does not apply the protocol's mint-decimal adjustment between the priced asset and the vault's `underlying_mint`, and the same issue exists in `reflect::read_price`.

```
>_ libraries/exponent_vaults_integrations/src/price_sources/chainlink.rs
```

```
RUST
```

```
pub fn read_price(
    last_updated_slot: u64,
    interface_accounts: &[Pubkey],
    accounts: &[AccountInfo],
    clock: &Clock,
) -> Result<Option<Number>> {
    [...]
    let exchange_rate = Number::from_ratio(price.value as u128, 10u128.pow(price.decimals as
        ↳ u32));

    Ok(Some(exchange_rate))
}
```

When `price_mint.decimals` is not equal to `underlying_mint.decimals`, token-to-base conversion will be off by `10(underlying_decimals - price_decimals)`, resulting in systematic AUM overstatement or understatement.

Remediation

Apply `decimal_adjustment_factor` before returning Chainlink and Reflect prices, matching the scaling behavior of price sources that already account for mint decimal differences.

Patch

Resolved in [PR#3780](#).

Stale Withdrawal Fill Capacity After Token Changes

HIGH

OS-EXV-ADV-06

Description

`queue_withdrawal` allocates each `WithdrawalAccount` with space based on `vault.token_entries.len()` at the time the withdrawal is queued. The account's fill vector, therefore, has capacity assumptions tied to the vault's token-entry set and destination token accounts. If the vault later adds token entries, or a token entry is remapped to a new `token_account_vault`, an older withdrawal may require recording additional `(mint, token_account_vault)` fill records that no longer fit in its fixed account size. As a result, existing withdrawals will become impossible to fill correctly.

```
>_ exponent_vaults/src/instructions/queue_withdrawal.rs
```

RUST

```
#[derive(Accounts)]
pub struct QueueWithdrawal<'info> {
    [...]
    #[account(
        init,
        payer = depositor,
        space = WithdrawalAccount::size_of(vault.token_entries.len()),
    )]
    pub withdrawal_account: Account<'info, WithdrawalAccount>,
    [...]
}
```

Remediation

Block token-entry mutations while queued withdrawals exist, or safely reallocate affected withdrawal accounts before additional fills are required.

Patch

Resolved in [PR#3799](#).

Lack of Foundational - Policy Guard on Policy Removal HIGH OS-EXV-ADV-07

Description

`apply_remove_policy` validates that the target policy is an existing Squads policy for the vault's settings, but it does not distinguish ordinary policies from foundational policies required for core vault operation. As a result, an authorized removal flow may execute `SettingsAction::PolicyRemove` against critical policy PDAs such as `FILL` or `TRANSFER` policies. If those policies are removed, later withdrawal-fill or asset-transfer flows may no longer satisfy Squads policy constraints, effectively disabling core vault operations. This is especially severe since the vault PDA cannot be re-added as a signer after initialization.

Remediation

Derive the expected foundational policy PDAs from their fixed seeds and check their expected seeds before executing removal.

Patch

Resolved in [PR#3808](#).

Permissionless Global Price Registry Initialization

MEDIUM

OS-EXV-ADV-08

Description

`initialize_prices` is currently permissionless but operates on a single global `ExponentPrices` PDA relied upon by the entire protocol. If an arbitrary caller initializes this account preemptively, they may assume the initial manager role and control critical setup state before the intended operator. Because all vault pricing depends on this singleton, the impact is protocol-wide rather than limited to a single vault.

```
>_ exponent_vaults/src/instructions/admin/initialize_prices.rs
```

RUST

```
pub fn initialize_exponent_prices_if_ready(ctx: &Context<InitializePrices>) -> Result<()> {  
    [...]  
    let mut exponent_prices = load_exponent_prices_mut(&exponent_prices_account)?;  
    if should_initialize || exponent_prices.price_managers_len == 0 {  
        exponent_prices.initialize(ctx.accounts.manager.key())?;  
    }  
  
    Ok(())  
}
```

Remediation

Limit access to an explicit trusted allowlist.

Patch

Resolved in [PR#3694](#).

Singleton Price Registry Reinitialization Takeover

MEDIUM

OS-EXV-ADV-09

Description

`initialize_prices::initialize_exponent_prices_if_ready` treats `price_managers_len == 0` as equivalent to uninitialized, even though those are not the same state. Thus, all managers were removed by a malicious or compromised price manager, a fully initialized `ExponentPrices` PDA may later reach `price_managers_len == 0`, while it still remains the live singleton price registry for the protocol.

```
>_ exponent_vaults/src/instructions/admin/initialize_prices.rs
```

RUST

```
pub fn initialize_exponent_prices_if_ready(ctx: &Context<InitializePrices>) -> Result<()> {  
    [...]   
    let mut exponent_prices = load_exponent_prices_mut(&exponent_prices_account)?;  
    if should_initialize || exponent_prices.price_managers_len == 0 {  
        exponent_prices.initialize(ctx.accounts.manager.key())?;  
    }  
  
    Ok(())  
}
```

At that point, the function will call `exponent_prices.initialize(ctx.accounts.manager.key())` for whoever invokes it next, effectively creating a reinitialization-takeover path for the protocol's global price registry by letting an arbitrary caller repopulate the manager set and take control of the `ExponentPrices` PDA. Since all vault pricing and price IDs depend on this PDA, they may disrupt existing price ID assumptions, and brick or redirect flows that depend on the canonical registry state.

Remediation

Disallow removing the last price manager and avoid treating an initialized zero-manager registry as safely reinitializable.

Patch

Resolved in [PR#3695](#).

Missing Mutable Vault in Proposal Lifecycle

MEDIUM

OS-EXV-ADV-10

Description

`activate_proposal`, `finalize_proposal`, and `cancel_proposal` all call methods that mutate `vault.proposal_vote_config.live_proposal_count`, but the `vault` account is not marked `mut` in their account constraints and as result, it is read-only. Consequently, `activate_live_proposal` and `decrement_live_proposal_count` may appear to run successfully, while their effects on `live_proposal_count` are silently lost after the instruction returns.

Remediation

Mark the `vault` account as `mut` in `activate_proposal`, `finalize_proposal`, and `cancel_proposal`.

Patch

Resolved in [PR#3696](#).

Unsafe Price Type Migration MEDIUM

OS-EXV-ADV-11

Description

`manage_prices::UpdatePrice` only updates the stored `price_type` byte on an existing `ExponentPrice` without migrating the full price definition. This is unsafe because each `PriceType` has different account requirements and pricing semantics, while the price ID may already be referenced by vault token entries or strategy positions. After migration, consumers may temporarily or permanently read a stale cached value produced by the old adapter under the new type label. `PT` / `YT` migrations are especially problematic because their pricing requires `implied_apy_bps`, which is only set during `AddPtPrice` / `AddYtPrice`. Thus, the lack of migration may result in used price IDs with stale or semantically invalid price data.

Remediation

Require fresh interface accounts, clear cached price state, reject unsupported `PT` / `YT` migrations, and validate migration safety before updating a price type.

Patch

Resolved in [PR#3765](#).

Utilization of Incorrect Confidence Bound in PythInverse MEDIUM OS-EXV-ADV-12

Description

In `pyth::read_price`, the inverse-price confidence is computed as

$1 / (\text{raw_price} - \text{raw_confidence}) - 1 / \text{raw_price}$, which measures the upside deviation of the inverse price when the raw price moves downward. However, the stale-price guard utilizes this confidence to tolerate negative moves in the computed inverse price, which occur when the raw price moves upward. For that direction, the relevant bound is the lower-side inverse deviation ($1 / \text{raw_price} - 1 / (\text{raw_price} + \text{raw_confidence})$). Using the upper-side inverse confidence overstates the tolerated negative inverse move and will result in the adapter retaining a stale higher cached price.

```
>_ libraries/exponent_vaults_integrations/src/price_sources/pyth.rs
```

```
RUST
```

```
pub fn read_price([...]) -> Result<Option<Number>> {  
    [...]  
    let (new_price, confidence) = if inverse {  
        require!(  
            raw_price > raw_confidence,  
            ErrorCodeVaults::InvalidPriceFeed  
        );  
        let inverse_price = Number::ONE  
            .checked_div(&raw_price)  
            .ok_or(ErrorCodeVaults::MathOverflow)?;  
        let inverse_confidence = Number::ONE  
            .checked_div(  
                &raw_price  
                .checked_sub(&raw_confidence)  
                .ok_or(ErrorCodeVaults::MathOverflow)?,  
            )  
            .ok_or(ErrorCodeVaults::MathOverflow)?  
            .checked_sub(&inverse_price)  
            .ok_or(ErrorCodeVaults::MathOverflow)?;  
        (adjust(inverse_price)?, adjust(inverse_confidence)?)  
    } else {  
        (adjust(raw_price)?, adjust(raw_confidence)?)  
    };  
    [...]  
}
```

Remediation

Compute inverse-side confidence against `raw_price + raw_confidence` for negative inverse moves.

Patch

Resolved in [PR#3768](#).

PT/YT Pricing Utilizes Linear Discounting

MEDIUM

OS-EXV-ADV-13

Description

`price_sources::pt::read_price` derives `PT` value with a simple linear discount formula, $PT = 1 / (1 + implied_apy * years_to_maturity)$. If the orderbook and CLMM components price the same `PT` / `YT` instruments using exponential discounting, then the shared vault price registry may disagree with execution-market valuation. This mispricing affects `PT` and `YT` AUM because valuations may be systematically biased by time to maturity and APY. It also affects flows that rely on the shared vault price registry.

```
>_ libraries/exponent_vaults_integrations/src/price_sources/pt.rs
```

RUST

```
pub fn read_price(
    last_updated_slot: u64,
    interface_accounts: &[Pubkey],
    accounts: &[AccountInfo],
    clock: &Clock,
    mint_decimals: Option<(u8, u8)>,
    implied_apy: f64,
    yt: bool,
) -> Result<Option<Number>> {
    [...]
    let seconds_to_maturity = (maturity_timestamp - current_timestamp) as f64;
    [...]
}
```

Remediation

Replace the linear formula with the same exponential `PT` / `YT` pricing model utilized by the orderbook and CLMM paths.

Patch

Resolved in [PR#3769](#).

Hylo Collateral Ratio Uses Pool Balance

MEDIUM

OS-EXV-ADV-14

Description

`hylo_cpi::state::stability_pool::compute_nav` calculates Hylo's collateral ratio using `amount_stablecoin`, which is the stablecoin balance held inside the stability pool, as the denominator. That is inconsistent with the protocol-level collateral-ratio definition, which should compare total `SOL` collateral value against total stablecoin supply. If the pool holds only a subset of outstanding stablecoins, using the pool balance makes the denominator too small and may materially overstate collateralization, which will result in overpriced Hylo LP positions in vault AUM.

```
>_ libraries/hylo_cpi/src/state/stability_pool.rs
```

RUST

```
pub fn compute_nav(
    hylo_data: &[u8],
    pyth_feed_data: &[u8],
    amount_stablecoin: UFix64<N6>, // hyusd token account balance
    amount_levercoin: UFix64<N6>,  // xsol token account balance
    stablecoin_supply: UFix64<N6>, // hyusd supply
    levercoin_supply: UFix64<N6>,  // xsol supply
    lp_token_supply: UFix64<N6>,  // shyusd supply
) -> Result<UFix64<N6>> {
    [...]
    let collateral_ratio = collateral_ratio(
        UFix64::<N9>::new(hylo.total_sol_cache.total_sol.bits),
        pyth_price.lower,
        amount_stablecoin,
    );
    [...]
}
```

Remediation

Utilize `stablecoin_supply` as the `collateral_ratio` denominator.

Patch

Resolved in [PR#3770](#).

Sentinel Escalation Ignores Withdrawal Carryover

MEDIUM

OS-EXV-ADV-15

Description

For weekly withdrawal schedules, `pending_withdrawal_backlog_due_at` contains carryover logic that pushes withdrawals created near the end of an eligible window into the next withdrawal window. `can_make_sentinel_manager`, however, bypasses that carryover path and computes escalation from `current_or_next_withdrawal_window_end(created_at)` plus the sentinel grace period.

```
>_ exponent_vaults/src/state/vault.rs
```

RUST

```
pub fn can_make_sentinel_manager(&self, created_at: u32, current_time: u32) -> Result<bool> {  
    [...]  
    let escalation_at = match self.withdrawal_period {  
        WithdrawalPeriodSettings::Interval { .. } => created_at  
            .checked_add(lock_time_seconds)  
            .and_then(|time| time.checked_add(SENTINEL_MANAGER_GRACE_PERIOD_SECONDS))  
            .ok_or(ErrorCodeVaults::MathOverflow)?,  
        WithdrawalPeriodSettings::WeeklySchedule {  
            days_of_week,  
            week_interval,  
            reference_timestamp,  
        } => Self::current_or_next_withdrawal_window_end(  
            created_at,  
            days_of_week,  
            week_interval,  
            reference_timestamp,  
        )  
            .and_then(|window_end| window_end.checked_add(SENTINEL_MANAGER_GRACE_PERIOD_SECONDS))  
            .ok_or(ErrorCodeVaults::InvalidWithdrawalPeriod)?,  
    };  
    Ok(current_time >= escalation_at)  
}
```

As a result, a withdrawal queued shortly before a window closes may be treated as missed for sentinel-escalation purposes even though the normal withdrawal backlog logic would not consider it overdue until the next eligible window. Therefore, a withdrawal created shortly before the window closes may enable a sentinel to obtain manager authority before there is a genuinely unserved executable withdrawal.

Remediation

Derive sentinel escalation from the sum of `pending_withdrawal_backlog_due_at(created_at)` and `SENTINEL_MANAGER_GRACE_PERIOD_SECONDS`, so both paths utilize the same carryover semantics.

Patch

Resolved in [PR#3778](#).

Earlier Opt-Out Withdrawal Deadline Ignored

MEDIUM

OS-EXV-ADV-16

Description

`add_pending_withdrawal` records `pending_withdrawal_backlog_due_at` only when `total_lp_pending_withdrawals` was previously zero, so later additions cannot move the global backlog deadline earlier. This is unsafe for approved opt-out withdrawals because their effective creation time may be backdated to `proposal.voting_ends_at`, making their computed backlog deadline earlier than withdrawals already pending in the vault. If such an opt-out withdrawal is added after a later normal withdrawal, the vault keeps the later deadline, and `blocks_manager_actions_for_pending_withdrawals` may continue returning false, resulting in manager actions to remain allowed even after the opt-out withdrawal is overdue.

```
>_ exponent_vaults/src/state/vault.rs
```

RUST

```
pub fn add_pending_withdrawal(&mut self, amount: u64, backlog_due_at: u32) -> Result<()> {
    let was_empty = self.total_lp_pending_withdrawals == 0;
    self.total_lp_pending_withdrawals = self
        .total_lp_pending_withdrawals
        .checked_add(amount)
        .ok_or(ErrorCodeVaults::MathOverflow)?;
    if was_empty {
        self.pending_withdrawal_backlog_due_at = backlog_due_at;
    }
    Ok(())
}
```

Remediation

Update `pending_withdrawal_backlog_due_at` to the minimum live backlog deadline whenever a new pending withdrawal is added.

Patch

Resolved in [PR#3781](#).

Opt-Out Votes Bypass Withdrawal Blocking

MEDIUM

OS-EXV-ADV-17

Description

`stake_vote::update_proposal_votes` accepts `VoteChoice::OptOut` and escrows LP for an opt-out path, but it does not check `vault.are_withdrawals_blocked()`. If opt-out votes later materialize into withdrawal accounts or withdrawal claims, this creates a bypass around the vault's global withdrawal-blocking flag, as ordinary `queue_withdrawal` rejects when withdrawals are blocked, while opt-out voting may still initiate an equivalent withdrawal-intent state.

```
>_ exponent_vaults/src/instructions/proposals/stake_vote.rs
```

RUST

```
fn update_proposal_votes(
    proposal: &mut ActionProposal,
    vote_choice: VoteChoice,
    lp_amount: u64,
) -> Result<()> {
    match vote_choice {
        [...]
        VoteChoice::OptOut => {
            proposal.opt_out_votes = proposal
                .opt_out_votes
                .checked_add(lp_amount)
                .ok_or(ErrorCodeVaults::MathOverflow)?;
        }
    }
    Ok(())
}
```

Remediation

Reject opt-out votes when withdrawals are blocked or create the opt-out withdrawal through a path that enforces the same flag.

Patch

Resolved in [PR#3786](#).

FillWithdrawal Leaves Post-Fill AUM Stale MEDIUM

OS-EXV-ADV-18

Description

`fill_withdrawal` refreshes `vault.financials.aum_in_base` before processing fills, but the handler mutates reserves and withdrawal state afterwards without performing a final AUM refresh. Because the token movements change vault reserve balances after the pre-fill AUM snapshot, the stored AUM may remain inconsistent with the actual post-fill asset state. Any later instruction that relies on `vault.financials.total_aum()` may then operate on stale values, including live-vault checks, max AUM supply enforcement, reserve/free-AUM calculations, and `verify_aum_change` deployment limits.

```
>_ exponent_vaults/src/instructions/admin/fill_withdrawal.rs
```

RUST

```
pub fn handler<'info>(<br>    mut ctx: Context<'_, '_, 'info, 'info, FillWithdrawal<'info>>,<br>    fills: Vec<FillParam>,<br>) -> Result<()> {<br>    [...]<br>    accounts<br>        .vault<br>        .recalculate_aum_in_base(&exponent_prices, aum_accounts)?;<br>    let management_fee_bps = accounts.vault.reserves_config.management_fee_bps;<br>    accounts.vault.financials accrue_management_fee(<br>        accounts.mint_lp.supply,<br>        management_fee_bps,<br>        Clock::get()?.unix_timestamp,<br>    )?;<br>    [...]<br>}
```

Remediation

Recalculate AUM after all fill-side balance and withdrawal updates have completed.

Patch

Resolved in [PR#3800](#).

Withdrawal Fees Rounded Away By Splitting

MEDIUM

OS-EXV-ADV-19

Description

`withdrawal::calculate_lp_fee_split` computes `fee_cut` as `floor(lp_amount * fee_bps / 10_000)`, so each individual fill may round fractional LP fees down to zero. Because `fill_withdrawal` applies this fee split independently for each filled withdrawal amount, a user may split a single economically large withdrawal into multiple smaller fills that each fall below the fee-rounding threshold. The aggregate fee then becomes lower than the fee that would have been charged on the same total LP amount in a single withdrawal, and in certain cases may be entirely avoided. This results in bypassing or materially reducing withdrawal fees.

```
>_ exponent_vaults/src/internal/withdrawal.rs
```

RUST

```
pub fn calculate_lp_fee_split(lp_amount: u64, fee_bps: u64) -> Result<(u64, u64)> {
    let fee_cut = lp_amount
        .checked_mul(fee_bps)
        .ok_or(ErrorCodeVaults::MathOverflow)?
        .checked_div(BASE_POINTS)
        .ok_or(ErrorCodeVaults::MathOverflow)?;

    let user_amount = lp_amount
        .checked_sub(fee_cut)
        .ok_or(ErrorCodeVaults::MathOverflow)?;

    Ok((fee_cut, user_amount))
}
```

Remediation

Utilize ceiling division for per-withdrawal fees or accumulate fractional fees across fills.

Patch

Resolved in [PR#3801](#).

Double-Incrementing of Kamino Reserve Price Usage MEDIUM OS-EXV-ADV-20

Description

In `manager_update_position`, `add_kamino_obligation_entry` and `upsert_kamino_obligation_reserve_price_mappings` call `validate_kamino_reserve_prices` before explicitly incrementing reserve-price pointers through `update_kamino_reserve_price_pointers`.

```
>_ exponent_vaults/src/instructions/admin/manager_update_position.rs
```

```
RUST
```

```
fn add_kamino_obligation_entry<'info>(  
    vault: &mut ExponentStrategyVault,  
    exponent_prices: &mut ExponentPrices,  
    remaining_accounts: &'info [AccountInfo<'info>],  
    entry: KaminoObligationEntry,  
) -> Result<()> {  
    [...]  
    validate_kamino_price(  
        &entry.quote_price_id,  
        exponent_prices,  
        vault.underlying_mint,  
    );  
    validate_kamino_reserve_prices(&entry, exponent_prices, remaining_accounts)?;  
    update_kamino_reserve_price_pointers(exponent_prices, &entry, true)?;  
    [...]  
}
```

Since `validate_price` inside the validation path already updates usage for each referenced `PriceId`, the later explicit increment counts the same reserve price a second time. This results in `ExponentPrices` usage counters exceeding the actual number of live references from vault positions. As a result, price may appear *in use* even after all real references are removed, preventing `remove_price` from succeeding, and global price accounting becomes inconsistent.

Remediation

Increment reserve price usage exactly once for each newly referenced reserve price.

Patch

Resolved in [PR#3804](#).

AUM - Sensitive Checks Use Stale Financials

MEDIUM

OS-EXV-ADV-21

Description

`wrapper_manage_vault_settings` decides whether the vault is live by reading `vault.financials.total_aum()` before recalculating AUM from current balances and positions. If the stored financials are stale, the wrapper may apply the wrong gating rule, where a live vault with outdated zero AUM may permit restricted settings changes, while an empty vault with stale nonzero AUM may reject valid setup actions. The same pattern is risky in other AUM-sensitive paths such as `SetMaxAumSupply`, direct policy-change authorization, and post-mutation reserve or deployment checks. Because AUM is derived state, relying on a cached value without a fresh recalculation may make authorization and invariant checks depend on outdated accounting rather than actual vault value.

```
>_ exponent_vaults/src/instructions/wrappers/wrapper_manage_vault_settings.rs
```

RUST

```
pub fn handler<'info>(<br>  ctx: Context<'_, '_, 'info, 'info, WrapperManageVaultSettings<'info>>,<br>  actions: Vec<VaultSettingsAction>,<br>) -> Result<()> {<br>  [...]<br>  if ctx.accounts.vault.financials.total_aum() > 0 {<br>    for action in &actions {<br>      match action {<br>        VaultSettingsAction::AddOrderbookEntry(_)<br>        | VaultSettingsAction::AddTokenAccountEntry(_)<br>        | VaultSettingsAction::AddYieldPositionEntry { .. }<br>        | VaultSettingsAction::AddClmmPositionEntry(_) => {}<br>        _ => return Err(ErrorCodeVaults::LivePositionSetupOnlySupportsAdditions.into()),<br>      }<br>    }<br>  }<br>  []<br>}
```

Remediation

Recalculate AUM immediately before AUM-sensitive checks and after AUM-changing settings or position mutations.

Patch

Resolved in [PR#3806](#).

MarketOffer Validation Omits AUM Diff

MEDIUM

OS-EXV-ADV-22

Description

`validate_raw_market_offer` and `validate_wrapper_market_offer` only validate that the market-offer destination is tracked, then return `ValidationResult::default()` with `aum_diff == 0`. However, a market offer may immediately transfer assets between `SY / PT / YT` balances or yield-position state, changing the vault's deployed/reserve AUM composition and potentially its net valuation. Since `validate_interaction` relies on `aum_diff` for deployment limits, reserve protection, or withdrawal-safety checks, treating the trade as AUM-neutral may understate the action's actual effect. Therefore, AUM-change limits and withdrawal-protection checks may be bypassed for these orderbook interactions.

Remediation

Compute and return the actual AUM delta for raw and wrapper market offers whenever tracked vault balances can change.

Patch

Resolved in [PR#3903](#).

Initial Token Entries Lack Account Validation

MEDIUM

OS-EXV-ADV-23

Description

`initialize_vault` accepts the initial `token_entries` vector directly from caller input and only validates that no two entries share the same mint. It does not verify that `token_squads_account` is actually owned by the expected Squads vault, that `token_account_vault` is owned by the vault PDA, or that either token account's mint matches `token_entry.mint`. As a result, vault initialization may permanently register arbitrary or mismatched token accounts inside core vault configuration, breaking later accounting and transfer flows.

```
>_ exponent_vaults/src/instructions/admin/initialize_vault.rs
```

RUST

```
pub fn handler<'info>([...]) -> Result<()> {  
    let proposal_config = proposal_vote_config.unwrap_or_default();  
    proposal_config.validate()?;  
    require!(  
        roles.manager.first() == Some(&manager),  
        ErrorCodeVaults::InvalidManager  
    );  
    [...]  
}
```

Remediation

Reject initial token entries or route all token entries through the same validated add-token-entry path.

Patch

Resolved in [PR#3807](#).

Wrapper Policy Updates Skip Configuration Validation MEDIUM OS-EXV-ADV-24

Description

`apply_wrapper_policy_action` forwards `PolicyConfig` directly into `apply_add_policy` and `apply_update_policy` without calling `policy_config.validate()`. The direct admin handlers perform this validation before invoking the same apply helpers, but wrapper/proposal-style flows may reach the shared apply layer without that precondition, resulting in the acceptance of invalid policy configurations. This weakens policy invariants and may result in malformed Squads policy state.

Remediation

Move validation into the shared apply helpers or call `policy_config.validate()` in every wrapper and proposal policy path.

Patch

Resolved in [PR#3809](#).

Locked Withdrawal AUM Uses Raw LP Supply

MEDIUM

OS-EXV-ADV-25

Description

`validate_interaction` computes withdrawal-protected AUM by calling `get_aum_locked_for_withdrawals`, using the raw LP mint supply as the denominator. That is inconsistent with vault accounting paths that use effective LP supply, which includes adjustments such as pending management-fee LP and other supply-side accounting effects.

```
>_ exponent_vaults/src/instructions/admin/validate_interaction.rs
```

RUST

```
pub fn handler<'info>(<br>  ctx: Context<'_, '_, 'info, 'info, ValidateInteractionHook<'info>>,<br>  inner_instructions: InnerInstructionsPayload,<br>  ) -> Result<()> {<br>  [...]<br>  if !inner_instructions.instructions.is_empty() {<br>    [...]<br>    let mut validation_view =<br>      hook_validation::build_vault_validation_view(vault, &exponent_prices)?;<br>    [...]<br>  }<br>  [...]<br>}
```

If raw supply is not aligned with the effective supply used for redemption math, the LP locked for queued withdrawals may be converted into too little base AUM. This understates the reserve amount that must remain protected for pending withdrawals, allowing managed interactions to pass `verify_aum_change` with less withdrawal coverage than intended.

Remediation

Pass the effective LP supply used by vault accounting into `get_aum_locked_for_withdrawals`.

Patch

Resolved in [PR#3810](#).

Permissionless Vault Initialization may Pin Shared Prices MEDIUM OS-EXV-ADV-26

Description

`initialize_vault` is permissionless, but during initialization it validates every `token_entry.price_id` against the shared global `ExponentPrices` registry. An arbitrary external vault creator may therefore permanently increase `positions_amount` for selected global price IDs simply by creating a vault that references them. Because price removal later checks whether a price is still in use, these attacker-created references may pin prices in the registry and prevent legitimate cleanup or force the global registry to retain attacker-chosen dependencies.

```
>_ exponent_vaults/src/instructions/admin/initialize_vault.rs
```

```
RUST
```

```
pub fn handler<'info>([...]) -> Result<()> {  
    [...]  
    for token_entry in token_entries.iter() {  
        token_entry.price_id.validate_price(  
            &mut exponent_prices,  
            token_entry.mint,  
            None,  
            underlying_mint,  
        )?;  
    }  
    [...]  
}
```

Remediation

Restrict vault creation to an explicit authorized creator allowlist governed by protocol admins.

Patch

Resolved in [PR#3828](#).

Direct Entrypoints Bypass Role Semantics

MEDIUM

OS-EXV-ADV-27

Description

The direct `manage_vault_settings` and `manager_update_position` handlers authorize execution by requiring `vault.to_account_info().is_signer`, but they do not independently enforce manager or allocator role membership. The wrapper paths add role semantics, such as allowing managers broader settings control while restricting allocators from role changes or other privileged mutations. If any internal path can cause the vault PDA to sign these direct instructions, it will bypass those wrapper-level role restrictions entirely. This creates an unsafe split between public instruction names and the intended manager/allocator authorization surface.

Remediation

Remove the externally exposed vault-signer-only path and make the canonical instruction names execute the wrapper role-checked flow.

Patch

Resolved in [PR#3829](#).

Allocator Wrappers Allow Non-Additive Live Changes MEDIUM OS-EXV-ADV-28

Description

`wrapper_manage_vault_settings` accepts both managers and allocators before applying only a narrow restriction that blocks non-managers from adding or removing role members. Outside that check, allocator callers may still access the broader settings and position mutation surface, including actions that are operationally sensitive or non-additive when the vault is not live. It does not restrict allocators to a precise allowlist of additive actions. Therefore, a compromised or overbroad allocator role may perform live non-additive changes outside the intended allocator responsibility.

```
>_ exponent_vaults/src/instructions/wrappers/wrapper_manage_vault_settings.rs
```

```
RUST
```

```
pub fn handler<'info>(<br>  ctx: Context<'_, '_, 'info, 'info, WrapperManageVaultSettings<'info>>,<br>  actions: Vec<VaultSettingsAction>,<br>  ) -> Result<()> {<br>    let is_manager = ctx<br>      .accounts<br>      .vault<br>      .validate_manager(&ctx.accounts.manager.key());<br>    let is_allocator = ctx<br>      .accounts<br>      .vault<br>      .roles<br>      .is_allocator(&ctx.accounts.manager.key());<br>    require!(<br>      is_manager || is_allocator,<br>      ErrorCodeVaults::UnauthorizedRole<br>    );<br>    [...]<br>  }
```

Remediation

Restrict allocator callers to an explicit additive settings and position update allowlist regardless of current AUM.

Patch

Resolved in [PR#3830](#).

Delegated Accounts Accepted As Withdrawal Sources MEDIUM OS-EXV-ADV-29

Description

`execute_withdrawal` and reserve withdrawal flows treat a token source account as valid if it is either directly owned by the vault PDA or merely delegated to the vault with sufficient allowance. This expands the trusted withdrawal-source surface beyond canonical vault reserve accounts and allows withdrawals to pull funds from arbitrary delegated token accounts. If the delegation is stale or misconfigured, the vault may unintentionally consume liquidity from accounts that were never meant to serve as withdrawal reserves.

Remediation

Require withdrawal source token accounts to be owned by the vault authority and reject delegated-only sources.

Patch

Resolved in [PR#3831](#).

Ability of Managers to Remove the Last Sentinel

MEDIUM

OS-EXV-ADV-30

Description

`manage_vault_settings::remove_role_member` enforces a nonempty manager set but applies no equivalent protection to the sentinel role. As a result, an authorized manager may remove the final sentinel and leave the vault without any emergency recovery authority. This is particularly dangerous because sentinels are intended to take over during overdue withdrawal situations or manager failures. Eliminating the last sentinel weakens the vault's operational safety guarantees and can permanently disable the intended fallback path during governance or withdrawal emergencies.

```
>_ exponent_vaults/src/instructions/admin/manage_vault_settings.rs
```

RUST

```
fn remove_role_member(
    vault: &mut ExponentStrategyVault,
    role: VaultRoleKind,
    member: Pubkey,
) -> Result<()> {
    let members = Self::role_members_mut(&mut vault.roles, role);
    let idx = members
        .iter()
        .position(|existing_member| *existing_member == member)
        .ok_or(ErrorCodeVaults::InvalidAccountData)?;
    members.remove(idx);
    require!(
        !members.is_empty() || role != VaultRoleKind::Manager,
        ErrorCodeVaults::InvalidAccountData
    );
    Ok(())
}
```

Remediation

Prevent removal of the final sentinel or require an atomic replacement so the vault always retains at least one active sentinel authority.

Patch

Resolved in [PR#3832](#).

Zero Mint Bypasses Price Validation LOW

OS-EXV-ADV-31

Description

`initialize_vault` permits a `TokenEntry` with `mint = Pubkey::default`, which is then forwarded as `sy_mint` to `price_id::validate_price`, effectively disabling the terminal mint check. That implies the price chain only has to be structurally valid. It no longer has to correspond to the reserve's actual asset mint. As a result, it becomes possible to register a fake reserve entry and attach an unrelated `PriceId`, which may distort AUM and LP pricing, block `PriceId` removal, and potentially trap real tokens during withdrawals.

```
>_ libraries/exponent_vaults_integrations/src/position_values /price_id.rs
```

RUST

```
pub fn validate_price(
    &self,
    exponent_prices: &mut ExponentPrices,
    sy_mint: Pubkey,
    base_mint: Option<Pubkey>,
    vault_underlying_mint: Pubkey,
) -> Result<> {
    [...]
    if sy_mint != Pubkey::default() {
        require_keys_eq!(
            last_entry.price_mint,
            sy_mint,
            ErrorCodeVaults::InvalidPriceMint
        );
    }
    [...]
}
```

Remediation

Reject default mints for all `TokenEntry` creation paths before validating or storing the entry.

Patch

The Exponent team acknowledged this issue.

Pre-Funded PDAs Block Account Creation LOW

OS-EXV-ADV-32

Description

`create_price_interface_accounts_account`, `create_mint_2022`, and `create_token_account` rely on `system_program::create_account` for deterministic PDA-style account creation. However, `create_account` expects the destination account to be uninitialized with zero lamports and no allocated data. Thus, if an attacker pre-funds the deterministic address with dust lamports, `create_account` will fail, blocking that seed. This creates a seed-specific denial of service for price interface account creation, mint creation, and token account creation.

Remediation

Tolerate pre-funded system accounts by topping up any rent shortfall, then utilize signed `allocate` and `assign` to initialize the deterministic account instead of unconditional `create_account`.

Patch

Resolved in [PR#3766](#) and [PR#3798](#).

AddPrice Accepts PT/YT Without APY Metadata

LOW

OS-EXV-ADV-33

Description

In `manage_prices`, `UpdatePriceAction::AddPrice` accepts an arbitrary `PriceType`, including `PriceType::Pt` and `PriceType::Yt`, but the generic variant has no `implied_apy_bps` field. `PT` / `YT` entries require that APY metadata to be stored in the `ExponentPrice` padding bytes for later discount-to-maturity pricing. As a result, `PT` / `YT` prices cannot be initialized correctly through the generic `AddPrice` variant.

```
>_ exponent_vaults/src/instructions/admin/manage_prices.rs
```

RUST

```
#[derive(AnchorSerialize, AnchorDeserialize, Clone)]
pub enum UpdatePriceAction {
    AddPrice {
        price_type: PriceType,
        price_mint: Pubkey,
        underlying_mint: Pubkey,
        interface_accounts: Vec<Pubkey>,
        price_interface_accounts_seed: [u8; 8],
    },
    [...],
}
```

Remediation

Merge `PT` / `YT` APY parameters into the common `AddPrice` flow or reject `PT` / `YT` explicitly from generic `AddPrice`.

Patch

Resolved in [PR#3767](#).

Failure to Back Active Proposal Votes by Eligibility Checkpoint LOW OS-EXV-ADV-34

Description

`stake_vote` measures voting power by transferring LP from the voter's live token account at vote time, rather than checking a proposal-specific stake snapshot. This allows newly minted or newly staked LP to vote on an already-active proposal. At the same time, activation snapshots for supply and rejection thresholds may be influenced by whether pending management-fee LP was collected before activation, creating a rejection threshold and voting set that are not tied to a stable governance-stake checkpoint.

```
>_ exponent_vaults/src/instructions/proposals/stake_vote.rs
```

```
RUST
```

```
pub fn handler<'info>(  
    ctx: Context<'_, '_, '_, 'info, StakeVote<'info>>,  
    vote_choice: VoteChoice,  
    lp_amount: u64,  
) -> Result<()> {  
    [...]  
    // Transfer LP tokens to escrow  
    ctx.accounts.do_transfer_lp(lp_amount)?;  
    [...]  
}
```

Remediation

Utilize governance-stake checkpoints with proposal eligibility lots so that only stake eligible before proposal activation can vote on that proposal.

Patch

The Exponent team acknowledged this issue.

Executed Proposals Bypass Opt-Out Withdrawals

LOW

OS-EXV-ADV-35

Description

`unstake_vote` only queues an opt-out withdrawal when `vote_choice == OptOut` and `proposal.status == Approved`. Once the approved proposal is executed and its status changes to `Executed`, the same opt-out voter no longer satisfies `should_queue_withdrawal` and instead receives LP back through the normal unstake path. Thus, an opt-out voter may unstake through the ordinary LP return path instead of preserving the opt-out withdrawal semantics.

```
>_ exponent_vaults/src/instructions/proposals/unstake_vote.rs
```

RUST

```
pub fn handler<'info>(ctx: Context<'_, '_, '_, 'info, UnstakeVote<'info>>) -> Result<()> {
    let timestamp = Clock::get()?.unix_timestamp;
    let lp_amount = ctx.accounts.vote_account.lp_amount;
    let vote_choice = ctx.accounts.vote_account.vote;
    let should_queue_withdrawal = vote_choice == VoteChoice::OptOut
    && ctx.accounts.proposal.status == ProposalStatus::Approved;
    [...]
}
```

Remediation

Treat both `Approved` and `Executed` proposal statuses as requiring opt-out withdrawal creation for `VoteChoice::OptOut`.

Patch

Resolved in [PR#3786](#).

Configuration Updates may Retroactively Extend Queued Withdrawals LOW

OS-EXV-ADV-36

Description

In `manage_vault_settings`, `set_withdrawal_period` and `set_withdrawal_cancel_cooldown` immediately overwrite the `withdrawal_period` or `withdrawal_cancel_cooldown_seconds` stored in `vault.reserves_config`.

```
>_ exponent_vaults/src/instructions/admin/manage_vault_settings.rs
```

RUST

```
fn set_withdrawal_period(
    vault: &mut ExponentStrategyVault,
    settings: WithdrawalPeriodSettings,
) -> Result<()> {
    vault.reserves_config.withdrawal_period = settings;
    vault.reserves_config.validate()?;
    Ok(())
}

fn set_withdrawal_cancel_cooldown(
    vault: &mut ExponentStrategyVault,
    withdrawal_cancel_cooldown_seconds: u32,
) -> Result<()> {
    vault.reserves_config.withdrawal_cancel_cooldown_seconds =
        withdrawal_cancel_cooldown_seconds;
    Ok(())
}
```

This is problematic as queued withdrawals derive their withdrawal executability and cancellation eligibility dynamically from those current settings, rather than persisting immutable execution or cancellation deadlines at creation time. Thus, a manager may retroactively increase lock durations or cancellation cooldowns after users have already queued withdrawals. This may delay users beyond the terms that applied when they queued.

Remediation

Store executable, cancellable, and sentinel-escalation timestamps on each withdrawal at queue time and use those fixed values thereafter.

Patch

The Exponent team acknowledged this issue.

Policy Manager Update Drops Existing Metadata

LOW

OS-EXV-ADV-37

Description

`manage_policy::update_policy_manager` is intended to change only the policy signer set so a sentinel may assume manager authority, but it rebuilds the Squads `PolicyUpdate` payload from stored `PolicyState`. During that reconstruction, `SpendingLimit` metadata such as `accumulate_unused` and some expiration semantics such as `PolicyExpiration::SettingsState` may not be preserved exactly. As a result, a sentinel-triggered signer replacement may silently alter the effective spending-limit policy, even though no policy configuration change was requested. This creates a configuration integrity risk during sentinel-managed policy updates.

```
>_ exponent_vaults/src/instructions/admin/manage_policy.rs
```

RUST

```
fn build_policy_manager_update_action(
    policy_key: Pubkey,
    vault_key: Pubkey,
    policy: squads_smart_account_program::accounts::Policy,
    sentinel: Pubkey,
) -> Result<SettingsAction> {
    let signers = replace_policy_manager_signers(vault_key, policy.signers, sentinel)?;
    require!(
        usize::from(policy.threshold) <= signers.len(),
        ErrorCodeVaults::InvalidPolicyForOperation
    );

    Ok(SettingsAction::PolicyUpdate {
        policy: policy_key,
        signers,
        threshold: policy.threshold,
        time_lock: policy.time_lock,
        policy_update_payload: policy_creation_payload_from_state(policy.policy_state)?,
        expiration_args: policy_expiration_to_args(policy.expiration),
    })
}
```

Remediation

Ensure to preserve all existing `SpendingLimit` metadata unless the policy configuration explicitly changes it.

Patch

Resolved in [PR#3805](#).

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-EXV-SUG-00	Multiple validation paths may be tightened to make account wiring, oracle configuration, governance state, and vault settings harder to misconfigure or misuse.
OS-EXV-SUG-01	Recommendations to separate routine price-manager permissions from manager-set administration to reduce the impact of compromised operational keys and to avoid using broad replacement flows for narrow parameter changes.
OS-EXV-SUG-02	Account-handling path may be refactored to avoid fragile manual indexing, stale pre-mutation sizing, and preventable rent or account-limit issues.
OS-EXV-SUG-03	Across the codebase, several low-risk accounting and pricing details should be made consistent to prevent repeated checkpoints, price-source calculations, and price-usage tracking from drifting over time.
OS-EXV-SUG-04	There are multiple instances of unnecessary, redundant, and duplicated code that should be addressed to improve readability and maintainability.
OS-EXV-SUG-05	Suggestions to ensure adherence to coding best practices.

Missing Validation Logic

OS-EXV-SUG-00

Description

1. `initialize_vault` loads `exponent_prices` as an unchecked account and utilizes it to validate every `TokenEntry.price_id`, but it does not verify that this account is the canonical `ExponentPrices` PDA. Add an Anchor PDA seed constraint for `[PRICES_SEED]` so the vault cannot be wired to an unexpected prices account.
2. Several price sources, including `pt`, `pyth`, `spl_stake_pool`, and `kamino_vault`, only validate that `interface_accounts` is non-empty, even though update-time logic assumes an exact account count. This may allow malformed price definitions with too few or too many interface accounts to be registered. Require each source to register the precise number of interface accounts it requires.

```
>_ libraries/exponent_vaults_integrations/src/price_sources/pt.rs
```

RUST

```
pub fn validate_interface_accounts(interface_accounts: &[Pubkey]) -> Result<()> {  
    require!(  
        !interface_accounts.is_empty(),  
        ErrorCodeVaults::MissingPriceAccount  
    );  
    Ok(())  
}
```

3. Price-source readers should add owner, discriminator, and cross-account consistency checks before reading external accounts, especially where deserialization currently trusts the supplied account data.
4. In `manager_update_position`, `add_offer_idx` should verify the target orderbook account/state still matches the configured `OrderbookEntry` before appending a new offer index; otherwise, stale or mismatched orderbook metadata may be tracked. Similarly, `add_token_account_balance` should validate the target `TokenAccountEntry` and supplied token account data are mutually consistent before mutation, ensuring strategy-position accounting cannot be extended with malformed or unrelated account references.
5. `WithdrawalPeriodSettings::WeeklySchedule` accepts any nonzero `days_of_week` bitmask and any `week_interval >= 1`, but it does not reject bits outside the seven valid weekday positions or validate the schedule anchor. Require the `days_of_week` bitmask to stay within the seven valid bits, and require `reference_timestamp` to be UTC-day aligned and not in the future.

```

>_ exponent_vaults/src/state/vault.rs RUST

pub fn validate(&self) -> Result<()> {
    match self {
        [...]
        WithdrawalPeriodSettings::WeeklySchedule {
            days_of_week,
            week_interval,
            ..
        } => {
            require!(*days_of_week != 0, ErrorCodeVaults::InvalidWithdrawalPeriod);
            require!(
                *week_interval >= 1,
                ErrorCodeVaults::InvalidWithdrawalPeriod
            );
            Ok(())
        }
    }
}

```

6. `prices::validate_exponent_prices_account_header` only checks the 8-byte discriminator before zero-copy casting the account data as `ExponentPrices`. If an upstream caller forgets to enforce the account owner or PDA seeds, a spoofed account with the correct discriminator but not owned by the `Exponent` program may be accepted by the shared loader. Enforce that the account owner is the `Exponent` program, so shared loaders remain safe even if an upstream caller omits an Anchor owner or seeds constraint.

```

>_ libraries/exponent_vaults_integrations/src/prices.rs RUST

fn validate_exponent_prices_account_header(_account: &AccountInfo<'_,>, data: &[u8]) ->
    Result<()> {
    require!(
        data[..8] == ExponentPricesMeta::DISCRIMINATOR[..],
        ErrorCodeVaults::InvalidAccountData
    );
    Ok(())
}

```

7. `StakeVote` creates `vote_account` with `init_if_needed`, but reuse must be explicitly constrained so an existing account cannot represent stale or incompatible vote state. Existing accounts should be reusable only for same-proposal, same-voter, same-side top-ups.
8. Yield-position, CLMM, and Kamino add/upsert helpers mutate durable strategy-position state, so their inner validation must be complete regardless of which entrypoint reaches them. Owner checks, expected mints, price-chain tails, and Kamino reserve mappings should be enforced in the shared add/upsert layer rather than relying on wrapper-only live checks.
9. `initialize_vault` seeds durable vault configuration, but several fields should be checked with the same invariants enforced by later settings-update paths. Initial reserve config, fee treasury, role

lists, role uniqueness/default keys, and proposal counters should be strictly validated at creation time so a vault cannot start in a state that later updates would reject.

10. Several `manage_vault_settings` setters mutate one `VaultConfig` field and then call `vault.reserves_config.validate()`, which revalidates the entire configuration. This makes targeted updates fragile, as an unrelated stale, legacy, or temporarily invalid field may result in an otherwise valid change to fail. Ensure each setter validates only the updated field and its direct dependencies instead of revalidating the full configuration.

```
>_ exponent_vaults/src/instructions/admin/manage_vault_settings.rs RUST

fn set_reserves_share(vault: &mut ExponentStrategyVault, reserves_share_bp: u32) ->
    Result<()> {
    require!(
        reserves_share_bp > 0 && u64::from(reserves_share_bp) < BASE_POINTS,
        ErrorCodeVaults::InvalidReservesShare
    );
    vault.reserves_config.reserves_share_bp = reserves_share_bp;
    vault.reserves_config.validate()?;
    Ok(())
}
```

11. In `manage_vault_settings`, `add_yield_position_entry` and `add_clmm_position_entry` manually inspect the last price ID to check that the price chain ends at the expected `PT` or `SY` mint, but then call `PriceId::validate_price` with `Pubkey::default()` as the input mint and check the tail mint manually afterwards. Instead, pass the expected tail mint directly into `validate_price` so price-chain mint validation and usage registration are enforced atomically by the shared price-validation routine.
12. `initialize_vault` currently accepts `mint_lp` and `token_lp_escrow` as unchecked mutable accounts and validates their PDA derivation inside helper functions. Move the deterministic PDA checks into Anchor seeds/bump account constraints where possible.

```
>_ exponent_vaults/src/instructions/admin/initialize_vault.rs RUST

pub struct InitializeVault<'info> {
    [...]
    /// CHECK: created & validated in handler
    #[account(mut)]
    pub mint_lp: UncheckedAccount<'info>,

    /// Holds the LP tokens escrow
    /// CHECK: created and validated in handler
    #[account(mut)]
    pub token_lp_escrow: UncheckedAccount<'info>,
    [...]
}
```

13. `manage_vault_settings::add_role_member` only checks that the member is not already present, so `Pubkey::default()` may be inserted into manager, curator, allocator, or sentinel role sets, even though it is not a valid operational authority. The function should reject `member == Pubkey::default()` before appending it.

```
>_ exponent_vaults/src/instructions/admin/manage_vault_settings.rs RUST

fn add_role_member(
    vault: &mut ExponentStrategyVault,
    role: VaultRoleKind,
    member: Pubkey,
) -> Result<()> {
    let members = Self::role_members_mut(&mut vault.roles, role);
    require!(
        !members.contains(&member),
        ErrorCodeVaults::InvalidAccountData
    );
    members.push(member);
    Ok(())
}
```

14. `manage_vault_settings::add_token_entry` silently calls `remove_conflicting_token_account_positions`, which deletes any existing `TokenAccount` strategy position for the same mint and releases its price references. It should instead reject when the mint is already tracked by a token-account strategy.
15. `manage_policy::execute_policy_action_flow` accepts an arbitrary `SettingsAction`, even though it is intended to execute only policy mutations. This broad type surface risks future misuse where unrelated Squads settings actions are routed through a helper that applies vault PDA approval and execution automatically, so the helper should restrict input to policy-specific Squad actions.
16. `squads::load_squads_settings` slices past the first 8 bytes and deserializes `SquadsSettingsData` without verifying that the account is owned by the Squads program or that its discriminator matches the expected settings account type. Add explicit owner and account-type discriminator checks before deserializing settings data.

```
>_ exponent_vaults/src/internal/squads.rs RUST

pub fn load_squads_settings(account: &AccountInfo<'_>) -> Result<SquadsSettingsData> {
    let data = account.try_borrow_data()?;
    let mut slice: &[u8] = &data[8..];
    SquadsSettingsData::deserialize(&mut slice).map_err(|_|
        ↳ ErrorCodeVaults::CustomError.into())
}
```

17. Multiple modules (such as `aum` and `token_account`) implement their own token-account parsing helpers, which increases the risk that one path omits owner checks or handles Token-2022

differently. These loaders should be consolidated into a shared helper that requires the account owner to be either SPL Token or Token-2022, before deserializing `TokenAccount`.

18. `vault::verify_aum_change` computes `effective_free_aum` with checked subtraction, so if `aum_locked_for_withdrawals > free_aum`, it fails with `MathOverflow` before reaching the intended reserve-limit check. It should instead use saturating subtraction to set `effective_free_aum` to 0, allowing the function to return the semantically correct `ReservesExceeded` when any projected deployed AUM exceeds available liquidity.

```

>_ exponent_vaults/src/state/vault.rs RUST

pub fn verify_aum_change(&self, aum_diff: i64, aum_locked_for_withdrawals: u64) ->
    Result<> {
    let free_aum = i128::from(self.get_free_aum()?);
    let locked = i128::from(aum_locked_for_withdrawals);
    let effective_free_aum = free_aum
        .checked_sub(locked)
        .ok_or(ErrorCodeVaults::MathOverflow)?;
    let current_in_positions = i128::from(self.financials.aum_in_base_in_positions);
    let new_in_positions = current_in_positions
        .checked_add(i128::from(aum_diff))
        .ok_or(ErrorCodeVaults::MathOverflow)?;
    require!(
        new_in_positions <= effective_free_aum,
        ErrorCodeVaults::ReservesExceeded
    );
    Ok(())
}

```

Remediation

Add the above validations at the lowest shared layer possible so that direct, wrapper, proposal, and SDK-driven paths all receive the same checks.

Patch

The above issues were resolved in [PR#3697](#), [PR#3773](#), [PR#3775](#), [PR#3776](#), [PR#3784](#), [PR#3785](#), [PR#3811](#), [PR#3814](#), [PR#3815](#), [PR#3816](#), [PR#3817](#), [PR#3820](#), [PR#3821](#), [PR#3822](#), [PR#3823](#), [PR#3824](#), [PR#3825](#), and [PR#3827](#).

Access Control and Administration Separation

OS-EXV-SUG-01

Description

1. `manage_prices` authorizes the caller only with `is_price_manager`, then allows that same role to execute both ordinary price-definition changes and `AddManager` / `RemoveManager`. This gives every price manager the ability to mutate the price-manager set, including adding new managers or removing existing ones, which is stronger than routine oracle/price administration. `AddManager` and `RemoveManager` should be restricted to a dedicated admin role, while price managers should remain limited to price lifecycle operations

```
>_ exponent_vaults/src/instructions/admin/manage_prices.rs
```

RUST

```
pub fn handler<'info>(<br>    ctx: Context<'_, '_, '_, 'info, ManagePrices<'info>>,<br>    actions: Vec<UpdatePriceAction>,<br>) -> Result<()> {<br>    [...]<br>    require!(<br>        exponent_prices.is_price_manager(manager_key)?,<br>        ErrorCodeVaults::InvalidPriceManager<br>    );<br>    let mut remaining_accounts_index = 0usize;<br>    for action in actions {<br>        match action {<br>            [...]<br>            UpdatePriceAction::AddManager { manager } => {<br>                exponent_prices.add_price_manager(manager)?;<br>            }<br>            UpdatePriceAction::RemoveManager { manager } => {<br>                exponent_prices.remove_price_manager(manager)?;<br>            }<br>            [...]<br>        }<br>    }<br>}
```

2. `implied_apy_bps` is stored as mutable `PT` / `YT` pricing metadata, but `manage_prices` only sets it during `AddPtPrice` / `AddYtPrice`. There is no targeted update path for changing it later. If this APY is intended to be adjusted over time, operators should have a controlled update path with appropriate access control and cache invalidation, rather than replacing the full price definition.

Remediation

Split privileged registry administration from ordinary price operations, and add explicit targeted update flows for dynamic price metadata.

Patch

1. Issue #1 resolved in [PR#3702](#).
2. Issue #2 resolved in `exponent-vaults-pending-audit` branch.

Code Refactoring

OS-EXV-SUG-02

Description

1. Several handlers manually track `remaining_accounts` with mutable index counters and direct slicing, which makes account consumption sensitive to layout changes and easy to desynchronize. As new accounts or action variants are added, stale index arithmetic may result in off-by-one bugs, wrong-account validation, or accidental reuse of accounts across actions. Utilize an iterator-based `next_account_info()` pattern in `execute_withdrawal`, `manage_prices`, `manage_vault_settings`, and `action_execution`.
2. The realloc helpers in `validate_interaction` and `action_execution` compute `current_rent` from the rent-exempt minimum for the current size rather than the account's actual lamports. Thus, if the vault already holds surplus lamports above the current rent-exempt minimum, the payer may be charged even though the account already has enough lamports to remain rent-exempt after growth. Ensure the funding calculation utilizes the account's actual lamport balance.
3. `initialize_squads_policy` and `initialize_squads_policy_transfer` create temporary Squads settings-transaction and proposal accounts, then execute them with `execute_settings_transaction`, without using the sync variant that closes the temporary settings-transaction and proposal accounts, leaving residual rent locked after vault initialization. The initialization flow should explicitly close temporary setup accounts.

```
>_ exponent_vaults/src/instructions/admin/initialize_vault.rs
```

RUST

```
fn initialize_squads_policy<'info>([...]) -> Result<()> {
    [...]
    let execute_cpi_ctx = CpiContext::new(squads_program.clone(), execute_cpi_accounts)
        .with_signer(signer_seeds)
        .with_remaining_accounts(vec![squads_policy.clone()]);

    squads_cpi::execute_settings_transaction(execute_cpi_ctx)?;
    Ok(())
}

fn initialize_squads_policy_transfer<'info>([...]) -> Result<()> {
    [...]
    let execute_cpi_ctx = CpiContext::new(squads_program.clone(), execute_cpi_accounts)
        .with_signer(signer_seeds)
        .with_remaining_accounts(vec![squads_policy.clone()]);

    squads_cpi::execute_settings_transaction(execute_cpi_ctx)?;
    Ok(())
}
```

4. `execute_withdrawal` trusts a caller-supplied `token_transfer_accounts_until` to delimit the withdrawal transfer accounts, even though the required span may be derived from `fills.len() * 2`, which would remove a redundant caller-controlled index. For large withdrawals, the design should also support partial or batched claims across multiple transactions, since a withdrawal with many fills may exceed Solana's transaction account limit.
5. `manage_vault_settings` and `manager_update_position` precompute account `realloc` size by simulating actions on cloned vault state before the real handler executes. If the dry-run logic differs from actual mutation semantics, especially for retain/remove logic and initially duplicated role members, the allocated size may be incorrect. Sizing should thus be derived from the actual post-mutation vault state.

Remediation

Prefer sequential account iteration, calculate rent from actual lamport balances, close temporary setup accounts, derive account spans from state where possible, and compute reallocation sizes after applying mutations.

Patch

The Exponent team acknowledged the issue regarding multi-transaction withdrawal claims, while all other issues were resolved in [PR#3774](#), [PR#3783](#), [PR#3788](#), [PR#3812](#), and [PR#3813](#).

Accounting and Price Source Consistency

OS-EXV-SUG-03

Description

1. `accrue_management_fee` advances `last_management_fee_accrued_at` before confirming that the elapsed interval produces a nonzero LP fee. If the computed fee rounds down to zero, the function returns without recording any fractional remainder, but the time interval has already been consumed. Repeated short-interval accruals can therefore permanently discard sub-unit fee accrual. The function should accumulate fractional fee debt.

```
>_ exponent_vaults/src/state/vault.rs
```

RUST

```
pub fn accrue_management_fee(
    &mut self,
    minted_supply: u64,
    management_fee_bps: u64,
    current_timestamp: i64,
) -> Result<u64> {
    [...]
    let elapsed = now
        .checked_sub(self.last_management_fee_accrued_at)
        .ok_or(ErrorCodeVaults::MathOverflow)?;
    self.last_management_fee_accrued_at = now;
    [...]
}
```

2. Price adapters should match the semantics of their underlying protocols and the project's generic oracle standards. `pt` pricing should use the intended 365-day year convention, `spl_stake_pool` pricing should validate the account owner and state before trusting deserialized pool data, and `pyth` confidence handling should follow the shared confidence/staleness policy.
3. In `prices`, `remove_price` bases removability only on the allocator register `PRICE_USAGE_AMOUNT`, while `ExponentPrice.positions_amount` is merely a mirrored cache. If the register and mirror diverge, the mirror will not prevent removal of a price that still appears referenced. Removal should require both counters to agree or eliminate the mirror so there is only one authoritative usage source.

Remediation

Preserve fractional accounting state, align each price source with its canonical implementation, and either remove mirrored usage fields or require all usage mirrors to agree before removing a price.

Patch

The Exponent team acknowledged the issue regarding sub-unit management fee accrual, while all other issues were resolved in [PR#3769](#), [PR#3776](#), [PR#3768](#), [PR#3787](#).

Redundant/Unutilized Code

OS-EXV-SUG-04

Description

1. In `prices::remove_price_manager`, the `len > 0` check is unreachable in practice because finding `idx` already proves the manager existed inside `self.price_managers[..len]`, which implies `len >= 1`. So the branch adds no safety and may be removed to avoid redundant defensive code.

```

>_ exponent_vaults_integrations/src/prices.rs                                typescript

pub fn remove_price_manager(&mut self, manager: Pubkey) -> Result<()> {
    let len = self.managers_len()?;
    let idx = self.price_managers[..len]
        .iter()
        .position(|pk| *pk == manager)
        .ok_or(ErrorCodeVaults::InvalidPriceManager)?;
    for i in idx..len.saturating_sub(1) {
        self.price_managers[i] = self.price_managers[i + 1];
    }
    if len > 0 {
        self.price_managers[len - 1] = Pubkey::default();
        self.price_managers_len = self
            .price_managers_len
            .checked_sub(1)
            .ok_or(ErrorCodeVaults::MathOverflow)?;
    }
    Ok(())
}

```

2. `initialize_vault` accepts `initial_lp_amount`, but the handler requires it to be zero and never uses it for minting or accounting. Remove the parameter unless non-zero mint-on-initialization support is implemented.

```

>_ exponent_vaults/src/instructions/admin/initialize_vault.rs                typescript

pub fn handler<'info>([...]) -> Result<()> {
    [...]
    require!(
        initial_lp_amount == 0,
        ErrorCodeVaults::InvalidInitialDepositState
    );
    [...]
}

```

3. In `proposal`, `VoteAccount::SIZE` includes a byte for an unstaked field that is not present in the structure, overestimating the required account space. Correct the oversized account constants.

4. In `vault`, `is_manager_invoke_blocked` and `is_user_invoke_blocked` expose status flags, but the flags are not enforced by manager or user invoke flows. Either enforce these flags consistently or remove them.
5. The `address_lookup_table` in `InitializeVault` is never utilized and may be removed. Also, `validate_tracked_token_account` and `validate_token_account_vault` helpers duplicate logic and should be consolidated into a single helper function. Furthermore, locked withdrawal LP should be tracked in a single variable, as currently `VaultFinancials` tracks locked withdrawal LP in both `lp_locked_for_withdrawal` and `total_lp_pending_withdrawals`.

Remediation

Remove all instances of unnecessary or redundant code.

Patch

The above issues were resolved in [PR#3695](#), [PR#3703](#), [PR#3782](#), [PR#3777](#), [PR#3818](#), [PR#3819](#), and [PR#3826](#).

Code Maturity

OS-EXV-SUG-05

Description

1. Promote `implied_apy_bps`, `price_decimals`, and `underlying_decimals` to explicit `ExponentPrice` fields makes the layout self-describing instead of relying on undocumented magic offsets in `_padding` and `reserved1`.
2. The current comment for `PriceId::Multiply` enumeration is mathematically correct, but it reads less naturally against the chain semantics enforced by `validate_price` and `calculate_price_inner`. Utilizing `SOL/USD price × LST/SOL` rate makes the conversion path clearer as first `LST` is expressed in `SOL`, then `SOL` is expressed in `USD`, which better matches how readers tend to reason about chained pricing.

```
>_ libraries/exponent_vaults_integrations/src/position_values /price_id.rs typescript

/// Price calculation method for token account and orderbook positions
#[derive(AnchorDeserialize, AnchorSerialize, Clone, Debug)]
pub enum PriceId {
    /// Single price reference
    Simple { price_id: u64 },
    /// Multiply prices together (e.g., LST/SOL rate × SOL/USD price)
    Multiply { price_ids: Vec<u64> },
}
```

Remediation

Implement the above-mentioned suggestions.

Patch

The Exponent team acknowledged these issues.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the [General Findings](#).

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
 - Improperly designed economic incentives leading to loss of funds.
-

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
 - Exploitation involving high capital requirement with respect to payout.
-

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
 - Forced exceptions in the normal user flow.
-

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.
-

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
 - Improved input validation.
-

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.